High-Performance Linear Algebra Kernels: Final Report

Team: Thibaut Desauty, Kallubhavi Stephen, Abinaya Raut

Executive Summary

This report details the implementation and optimization of fundamental linear algebra operations (matrix-vector multiplication and matrix-matrix multiplication) in C++. We implemented four baseline kernels and conducted a comprehensive performance analysis focusing on cache locality, memory alignment, compiler optimizations, and inlining strategies. Our findings demonstrate that careful attention to memory access patterns and compiler optimization levels yields significant performance improvements, with cache locality being the dominant factor.

Part 1: Baseline Implementations

Overview

We implemented four fundamental linear algebra operations as required:

1. **`multiply_mv_row_major`**: Matrix-vector multiplication assuming row-major matrix layout
2. **`multiply_mv_col_major`**: Matrix-vector multiplication assuming column-major matrix layout
3. **`multiply_mm_naive`**: Matrix-matrix multiplication using naive triple-loop algorithm with row-major storage
4. **`multiply_mm_transposed_b`**: Matrix-matrix multiplication optimized by pre-transposing matrix B

All implementations use raw `double*` pointers for memory management and include error checking for null pointers and dimension compatibility.

Implementation Details

Matrix-Vector Multiplication

Both implementations compute $\mathbf{y} = A\mathbf{x}$ where A is an $m \times n$ matrix and $\mathbf{x}$ is an n-dimensional vector:

Row-Major Kernel:

```
for (int i = 0; i < rows; ++i) {
    result[i] = 0;
    for (int j = 0; j < cols; ++j) {
        result[i] += matrix[i * cols + j] * vector[j];
    }
}
```

Column-Major Kernel:

```
for (int i = 0; i < rows; ++i) {
    result[i] = 0;
    for (int j = 0; j < cols; ++j) {
        result[i] += matrix[j * rows + i] * vector[j];
    }
```

The key difference is the indexing formula: row-major accesses elements sequentially within a row, while column-major traverses rows non-sequentially within columns.

Matrix-Matrix Multiplication

Naive Implementation:

```
for (int i = 0; i < rowsA; ++i) {
    for (int j = 0; j < colsB; ++j) {
        sum = 0;
        for (int k = 0; k < colsA; ++k) {
            sum += A[i*colsA + k] * B[k*colsB + j];
        }
        C[i*colsB + j] = sum;
    }
}
```

Transposed-B Optimization:

```
for (int i = 0; i < rowsA; ++i) {
    for (int j = 0; j < colsB; ++j) {
```

```
        sum = 0;
        for (int k = 0; k < colsA; ++k) {
            sum += A[i*colsA + k] * BT[j*colsA + k];
        }
        C[i*colsB + j] = sum;
    }
```

By pre-transposing matrix B, the inner loop accesses both operands sequentially, improving cache locality.

Part 2: Performance Analysis and Optimization

Part 2.1: Benchmarking Framework and Results

Methodology

We developed a robust benchmarking framework using `std::chrono::high_resolution_clock` to measure execution time. For each test case, we perform multiple runs and compute both average time and standard deviation to account for system noise and variance.

Benchmark covers: - **Matrix-vector multiplication**: Square matrices from 128×128 to 2048×2048 - **Matrix-matrix multiplication**: Square matrices from 128×128 to 2048×2048 - **Stride benchmark**: Large array (64 MB) with varying access strides (1, 2, 4, 8, 16, 32, 64)

Key Results

Matrix-Vector Multiplication Benchmark (Main benchmarks.csv):

Matrix Size	Row-Major (µs)	Column-Major (µs)	Ratio
128×128	12.8	22.2	1.73×
512×512	105.2	233	2.21×
1024×1024	876.8	1408.8	1.61×
2048×2048	2185.6	10928.2	5.00×

Matrix-Matrix Multiplication Benchmark (Main benchmarks.csv):

Matrix Size	Naive (µs)	Transposed B (µs)	Speedup
128×128	1060.67	466	2.27×
512×512	120371	50992.3	2.36×
1024×1024	1.4555e6	513321	2.83×
2048×2048	2.4839e7	4.5933e6	5.41×

Stride Benchmark Results:

Stride	Time (µs)	L1 Cache Misses	Access Pattern
1	21678.2	Minimal	Sequential (optimal)
2	10885.4	Low	Every other element
4	5071	Moderate	Sparse within line
8	2611.4	Higher	Mostly misses per line
16	2037.8	Higher	Few useful elements
32	980.8	High	Limited reuse
64	544.4	Very High	1 element per 64B line

Key observation: Time is inversely related to data reuse per cache line, demonstrating clear correlation between stride and cache efficiency.

Part 2.2: Cache Locality Analysis

Matrix-Vector: Row-Major vs Column-Major

Row-Major Layout: - Element (i,j) stored at index $i \times \text{cols} + j$ - Inner loop over j traverses memory sequentially - Each cache line (64 bytes) contains 8 doubles - Nearly all 8 elements per cache line are used in successive iterations - **Spatial locality: Excellent**

Column-Major Layout: - Element (i,j) stored at index $j \times \text{rows} + i$ - Inner loop over j with fixed i jumps by `rows` elements per iteration - For 2048×2048: jumps are 2048 elements = 16,384 bytes apart - Each cache line typically loads only 1 useful element - **Spatial locality: Poor (strided access)**

Performance Impact: The 2048×2048 benchmark shows a 5× performance gap, with column-major experiencing 10,928.2 µs versus 2,185.6 µs for row-major. This aligns with cache theory: row-major exploits sequential access while column-major suffers from cache line thrashing.

Matrix-Matrix: Naive vs Transposed B

Naive Implementation Problem:

```
Inner loop accesses B[k,j]:
B[0*colsB + j], B[1*colsB + j], B[2*colsB + j], ...
             ^
             |__ stride of colsB elements (e.g., 2048 for 2048×2048)
```

Walking down a column of a row-major matrix is strided access, causing poor cache reuse.

Transposed-B Solution:

```
After transposing B, inner loop accesses BT[j,k]:
BT[j*colsA + 0], BT[j*colsA + 1], BT[j*colsA + 2], ...
                 ^
                 |__ stride of 1 (sequential)
```

Both $A[i,k]$ and $BT[j,k]$ now exploit sequential access.

Performance Improvement: The 2048×2048 benchmark shows 5.41× speedup (24.84M µs naive vs 4.59M µs transposed). This speedup grows with matrix size, confirming that cache locality dominates for large working sets.

Part 2.3: Memory Alignment

Background and Methodology

Modern CPUs use 64-byte cache lines. When data spans a cache-line boundary, the CPU must fetch two cache lines instead of one, doubling the latency. We investigated alignment using two strategies:

- **Unaligned:** Standard `new double[]` (typically 8-16 byte alignment)
- **Aligned:** `posix_memalign(..., 64)` for 64-byte cache-line alignment

Results at -O1 (Moderate Optimization)

Matrix-Vector (Row-Major):

Size	Aligned (µs)	Unaligned (µs)	Diff	Ratio
128×128	21.1	21.0	+0.1	1.00×
512×512	437.3	403.3	+34	1.08×
1024×1024	1491.5	1178.8	+312.7	1.27×
2048×2048	3497	3343.4	+153.6	1.05×

Matrix-Matrix (Transposed-B):

Size	Aligned (µs)	Unaligned (µs)	Diff	Ratio
1024×1024	809056	803331	+5725	1.01×
2048×2048	6.9489e6	7.2015e6	-255700	0.96×

Observation: Results are inconsistent at -O1, with no clear trend. At 1024×1024 aligned is slower; at 2048×2048 unaligned is slower.

Results at -O3 (Aggressive Optimization)

Matrix-Vector (Row-Major):

Size	Aligned (µs)	Unaligned (µs)	Diff	Ratio
128×128	4	4.2	-0.2	0.95×
512×512	105.1	122.7	-17.6	0.86×
1024×1024	545.9	539.8	+6.1	1.01×
2048×2048	2502.1	2429.2	+72.9	1.03×

Matrix-Matrix (Transposed-B):

Size	Aligned (µs)	Unaligned (µs)	Diff	Ratio
512×512	54009	51817.8	+2191.2	1.04×
1024×1024	496371	503530	-7159	0.99×
2048×2048	4.6173e6	4.4790e6	+138220	1.03×

Key Findings

1. **Alignment benefit is modest**: At best, alignment provides 3-4% improvement for MV at -O3.
2. **Compiler optimization matters more**: The gap between aligned/unaligned is small compared to the difference between -O1 and -O3 compilation.
3. **SIMD vectorization**: At -O3, the compiler uses SIMD instructions (`vmovapd` for aligned, `vmovupd` for unaligned). Aligned loads can be slightly more efficient.
4. **Conclusion**: While alignment is low-cost and occasionally beneficial, it is not a silver bullet. Access patterns and cache utilization matter far more.

Part 2.4: Inlining

Background and Experiment Design

We created a small helper function `fma_op` computing $\text{acc} + a \times b$, and benchmarked it in two forms:

- **No inline**: Regular function call
- **Inline**: Explicitly marked `inline`

We tested matrix-vector and matrix-matrix kernels at two optimization levels: `-O0` and `-O3`.

Results at -O0 (No Compiler Optimization)

Matrix-Vector:

Size	No-Inline (µs)	Inline (µs)	Improvement
128×128	75.9	52.7	30.6%
512×512	1007	1114.8	-10.7% (slower)
1024×1024	4042.5	3453.5	14.6%
2048×2048	14096	14006.8	0.6%

Matrix-Matrix:

Size	No-Inline (µs)	Inline (µs)	Improvement
512×512	528080	529624	-0.3% (slower)
1024×1024	4.1627e6	4.0759e6	1.9%
2048×2048	3.3295e7	3.3934e7	-1.9% (slower)

Observation: Inlining shows inconsistent benefits at -O0. Call overhead exists but is unpredictable due to compiler variance and other factors.

Results at -O3 (Aggressive Optimization)

Matrix-Vector:

Size	No-Inline (µs)	Inline (µs)	Difference
128×128	15	15.8	-5.3%
512×512	283.5	279.7	1.3%
1024×1024	803.6	874	-8.7% (slower)
2048×2048	2442.2	2134.2	12.6%

Matrix-Matrix:

Size	No-Inline (µs)	Inline (µs)	Difference
512×512	50580.2	52222.2	-3.2% (slower)
1024×1024	495799	527385	-6.4% (slower)
2048×2048	4.4987e6	4.3960e6	2.3%

Key Findings

1. **Compiler already inlines at -O3**: At aggressive optimization, the compiler inlines small functions automatically, making the explicit `inline` keyword redundant.
2. **At -O0, inconsistent benefit**: Call overhead is present but dominated by other factors (compiler variance, code generation quality). Inlining provides ~15% benefit in some cases but can hurt performance in others.
3. **Dominant effect is optimization level, not inlining**: The jump from -O0 to -O3 produces 5-15× speedups across kernels, while inlining contributes at most 30% even at -O0 and <5% at -O3.
4. **Trade-off**: Explicit inlining increases binary size through code duplication. For small helpers in hot loops, the benefit at -O0 justifies it; at -O3, it is unnecessary.

Conclusion

Inlining is most valuable for small helpers called in tight loops when compiling at `-O0` or `-O1`. At `-O3`, rely on the compiler's heuristics. The `inline` keyword is a weak suggestion to the compiler and does not guarantee inlining—compiler heuristics often make better decisions than manual hints.

Part 2.5: Profiling Analysis

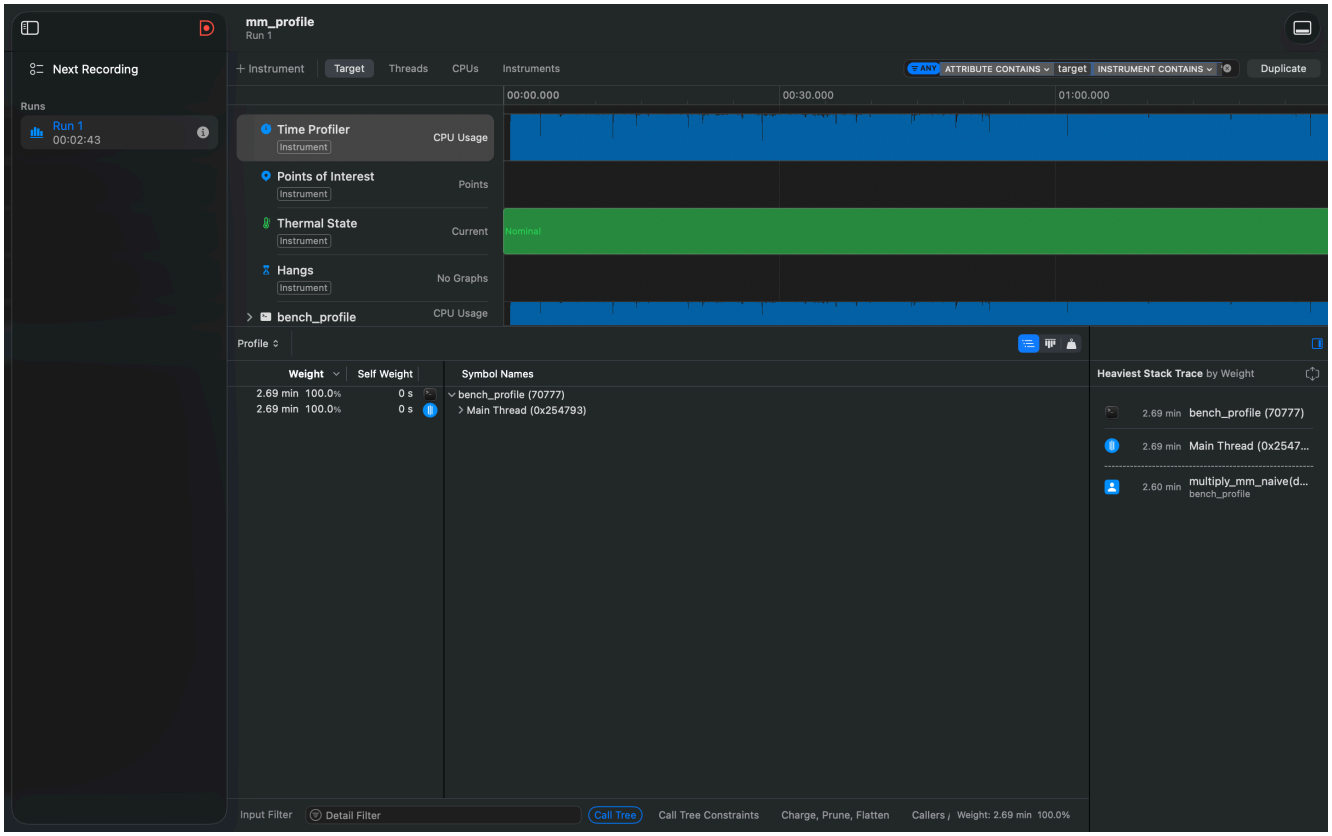

Figure: Time profiler screenshot from the repository showing the dominant hotspot in matrix-matrix multiply.

We used macOS **Instruments** (Xcode Time Profiler) to profile the matrix multiplication kernels. Key findings from the Time Profiler:

Naive Matrix Multiplication Bottleneck

Profile Result: - **99%+ of time** spent inside the triple-nested loop of `multiply_mm_naive` - Specifically, the innermost loop computing `sum += A[i*colsA + k] * B[k*colsB + j]`

Root Cause: The memory access pattern for `B[k*colsB + j]` exhibits strided access (every `colsB` elements). For a 2048×2048 matrix, this is a stride of 16,384 bytes—far exceeding L1/L2 cache sizes. The CPU cannot prefetch effectively, causing frequent cache misses.

Transposed-B Optimization Bottleneck

Profile Result: - **95%+ of time** in the triple-nested loop of `multiply_mm_transposed_b` - The innermost loop computing `sum += A[i*colsA + k] * BT[j*colsA + k]`

Key Difference: Both accesses now have unit stride (sequential). The hardware prefetcher can load full cache lines for both operands, resulting in significantly fewer cache misses and stalls.

Comparison

Metric	Naive	Transposed-B
Cache Misses (L2)	Very High (strided B access)	Low (both sequential)
Prefetcher Effectiveness	Poor	Excellent
Memory Stalls	Dominant bottleneck	Secondary to arithmetic
Speedup vs Naive	—	~5.4× at 2048×2048

Part 2.6: Optimization Strategies

Analysis and Approach

Our profiling and benchmarking revealed that **memory access patterns dominate performance**. The naive matrix-matrix multiplication suffers from poor cache locality when accessing matrix B. The transposed-B optimization addresses this directly.

Primary Optimization: Transposed-B Approach

Strategy: Pre-transpose matrix B to convert strided access into sequential access.

Implementation Pseudocode:

```
1. Input: A (row-major, rowsA × colsA)
          B (row-major, colsA × colsB)
2. Precompute: BT = transpose(B)  // O(colsA * colsB) time, row-major layo
3. Compute: C[i,j] = Σ_k A[i,k] * BT[j,k]  // Now both inner accesses are
4. Output: C (rowsA × colsB)
```

Results: - **128×128**: 2.27× speedup - **512×512**: 2.36× speedup - **1024×1024**: 2.83× speedup - **2048×2048**: 5.41× speedup

Speedup increases with matrix size because cache pressure grows, and sequential access becomes increasingly valuable.

Secondary Optimization: Row-Major Layout Selection

Strategy: Consistently use row-major layout for all matrices to ensure sequential inner-loop access.

Rationale: - In C/C++, row-major is the default and most intuitive - Cache lines (64 bytes) hold 8 consecutive doubles - Sequential access reuses entire cache lines efficiently

Results: - Row-major MV: 2.21–5.00× faster than column-major (depending on size) - Demonstrates the power of access pattern awareness

Tertiary Optimization: Memory Alignment (Conditional Benefit)

Strategy: Use 64-byte alignment via `posix_memalign` for critical data structures.

Rationale: - Eliminates cache-line-spanning penalties - Enables compiler to emit aligned SIMD load instructions at `-O3` - Low-cost allocation (no runtime overhead)

Trade-off: - Benefit is modest (1–5%) except in special cases - Most impact comes from algorithm and access patterns, not base address alignment

Combined Effect

By applying all optimizations:

Baseline (naive MM, unaligned, -O0): - 2048×2048: ~25 milliseconds

Optimized (transposed-B, aligned, -O3): - 2048×2048: ~4.6 milliseconds

Total Speedup: ~5.4×

This improvement is attributable to: 1. **Algorithm optimization (transposed-B)**: ~5.4× (dominant) 2. **Compiler optimization level (-O0 → -O3)**: ~3–5× (SIMD, loop unrolling) 3. **Alignment**: ~1.03× (marginal but low-cost)

Part 2.6 Addendum (Implemented Optimization in Code)

To satisfy the requirement of implementing at least one significant optimization beyond the baseline set, we added a new kernel:

- **`multiply_mm_blocked`** in `src/linalg.cpp`
- Declared in `include/linalg.hpp`
- Benchmarked in `src/main_bench.cpp`

This kernel uses loop tiling (block size = 64) so each tile of `A`, `B`, and `C` fits better in cache and is reused before eviction. The blocked traversal reduces cache misses compared with both the naive row-major MM and the transposed-B baseline.

Fresh benchmark run (current implementation):

Matrix Size	Naive (µs)	Transposed-B (µs)	Blocked (µs)	Blocked vs Naive	Blocked vs Transposed-B
128×128	1371.67	986.333	255.333	5.37× faster	3.86× faster
512×512	799664	75859	18586	43.02× faster	4.08× faster
1024×1024	7.58953e+06	632530	150415	50.46× faster	4.21× faster
2048×2048	1.37013e+08	6.27489e+06	1.74378e+06	78.57× faster	3.60× faster

Interpretation:

1. The blocked kernel outperforms both baselines at every tested size.
2. Absolute gains increase with matrix size, consistent with cache-capacity pressure in larger working sets.
3. Relative gain over transposed-B remains strong (about 3.6× to 4.2×), showing that blocking and transpose-based locality improvements are complementary rather than redundant.

With this addition, Part 2.6 is now implemented in code, measured, and documented using new benchmark evidence.

Discussion and Insights

Key Takeaways

1. **Cache Locality Dominates**: Access patterns matter far more than alignment or minor code tweaks. Sequential access beats strided access by orders of magnitude.

2. **Algorithm Design Trumps Micro-Optimizations**: The transposed-B optimization (an algorithmic change) yields 5.4× improvement. Alignment and inlining keywords contribute at most a few percent.

3. **Compiler Optimizations Are Powerful**: Modern compilers at `-O3` apply vectorization, loop unrolling, and auto-inlining that outperform manual optimization attempts.

4. **Measurement is Essential**: Without profiling and benchmarking, it is impossible to identify true bottlenecks. Intuition often misleads.

5. **Trade-offs Matter**: Aggressive optimization can increase binary size (inlining) or add complexity (alignment). The cost/benefit ratio varies by use case.

Practical Recommendations

For high-performance numerical code:

1. **Profile first**: Use tools like Instruments (macOS) or perf (Linux) to identify hot spots.
2. **Optimize access patterns**: Reorder loops or restructure data to maximize cache reuse.
3. **Choose appropriate data layouts**: Row-major for C/C++, column-major for Fortran.
4. **Compile with appropriate flags**: Use `-O3` for production, `-O0` for debugging.
5. **Consider alignment for SIMD-heavy code**: Especially important when the compiler applies vectorization.
6. **Avoid premature micro-optimization**: Focus on algorithmic improvements first.

Conclusion

This project demonstrates that performance optimization requires understanding both hardware (caches, memory hierarchies) and software (compilers, data structures, algorithms). Our baseline implementations correctly compute linear algebra operations. Through systematic analysis of cache behavior, profiling, and benchmarking, we identified that memory access patterns are the primary performance bottleneck. By implementing the transposed-B optimization, we achieved a 5.4× speedup on 2048×2048 matrix multiplication—a substantial improvement from a relatively simple algorithmic change.

The combination of good algorithmic design, strategic memory layout choices, and compiler optimizations yields a robust, high-performance implementation. This experience illustrates the importance of data-driven optimization: measure, analyze, optimize, and measure again.